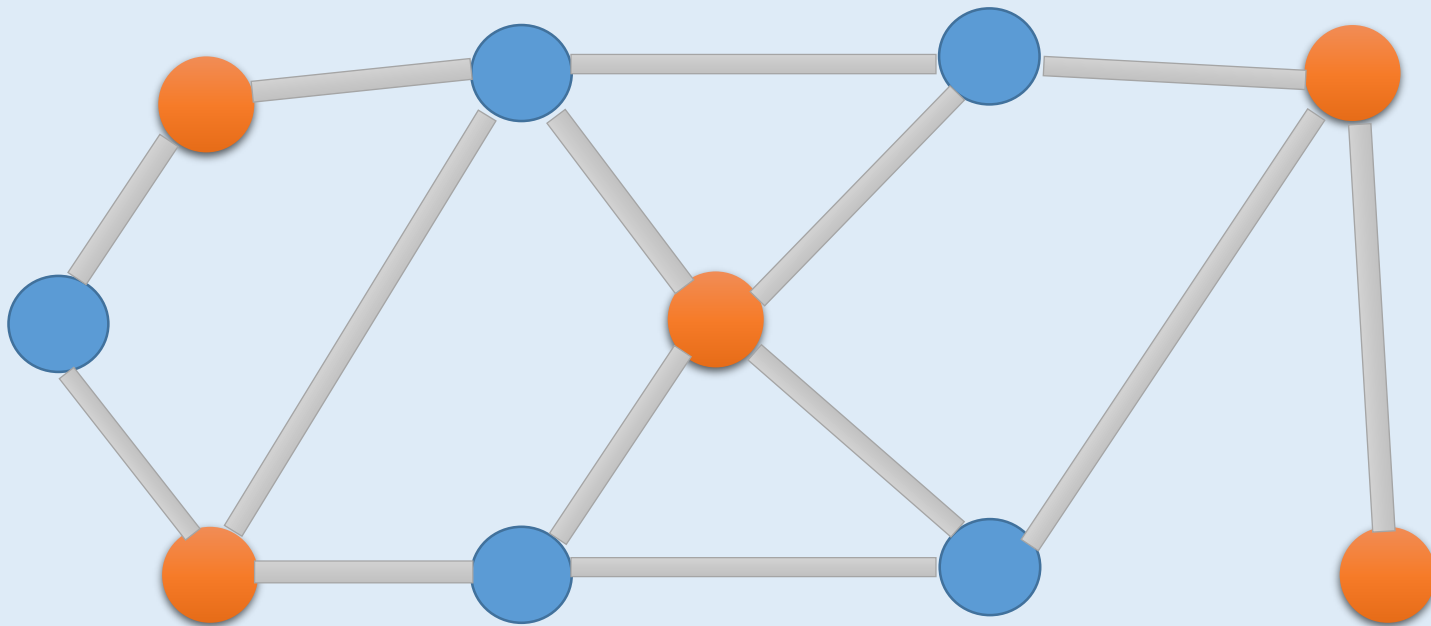


VERTEX COVER PROBLEM



APPROXIMATION ALGORITHMS

➤ Definition: Approximation algorithm

- An approximation algorithm for a problem is a *polynomial-time algorithm* that, when given input i , outputs an element of $FS(i)$.

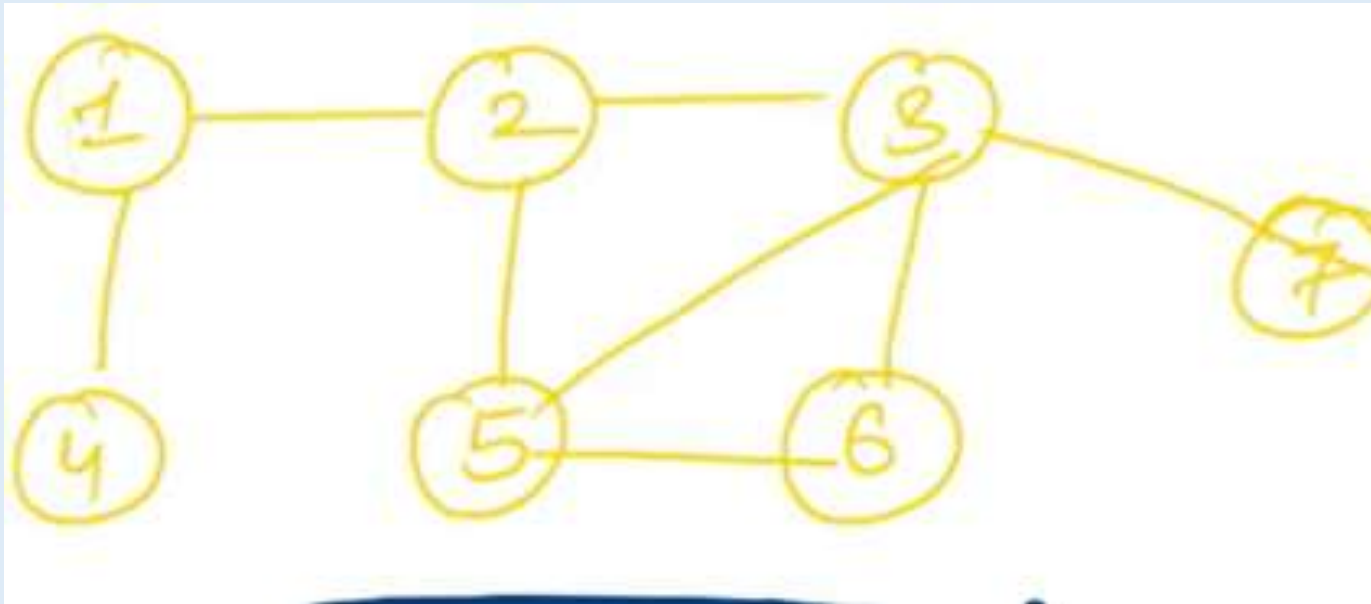
➤ Feasible solution set

- A feasible solution is an object of the right type but not necessarily an optimal one.

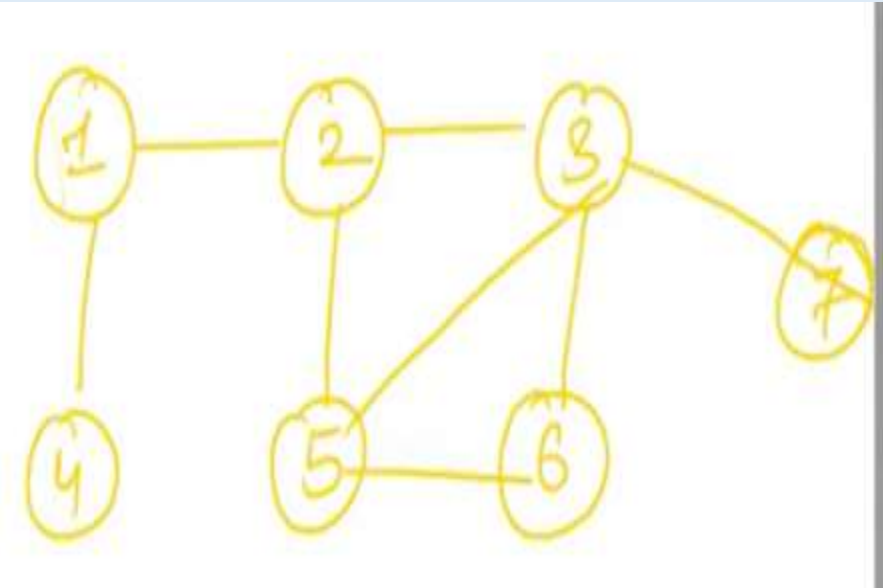
$FS(i)$ is the set of feasible solutions for i .

Vertex Cover Problem

- In the mathematical discipline of graph theory, “A *vertex cover* (sometimes node cover) of a graph is a subset of vertices which “*covers*” every edge.
- An edge is *covered* if one of its endpoint is chosen.
- In other words “A *vertex cover* for a graph G is a set of vertices incident to every edge in G .”
- The *vertex cover problem*: What is the minimum size vertex cover in G ?



Goal: Find the vertices that covers maximum edges of the graph.

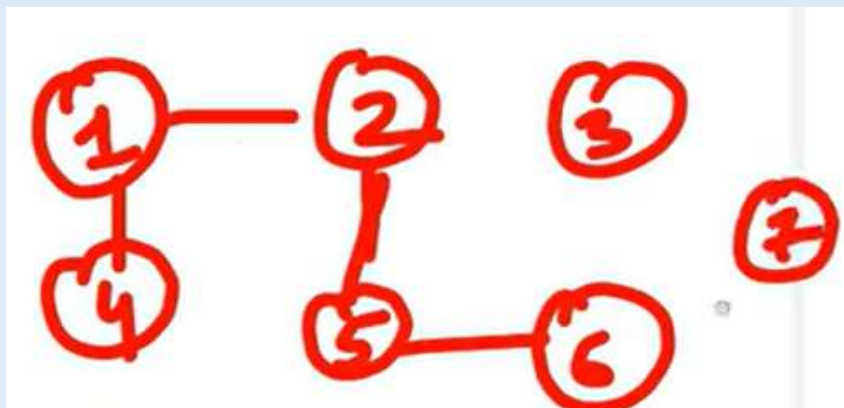


Vertex	Connected vertices	No. of Connections
1	[2,4]	2
4	[1]	1
2	[3,5,1]	3
5	[2,6,3]	3
3	[2,7,5,6]	4
6	[3,5]	2
7	[3]	1

Maximum Edges covered vertex: 3

One of the min subset is :- $\{3,$

Next step: Exclude all the edges connected with 3 and create the graph accordingly



Vertex	Connected vertices	No. of Connections
1	[2,4]	2
4	[1]	1
2	[3,5,1]	3
5	[2,6,3]	3
3	[2,7,5,6]	4
6	[3,5]	2
7	[3]	1

Next Step: Again, find the vertices which cover max edges of the Graph

1 [2,4] → 2

4 → 1

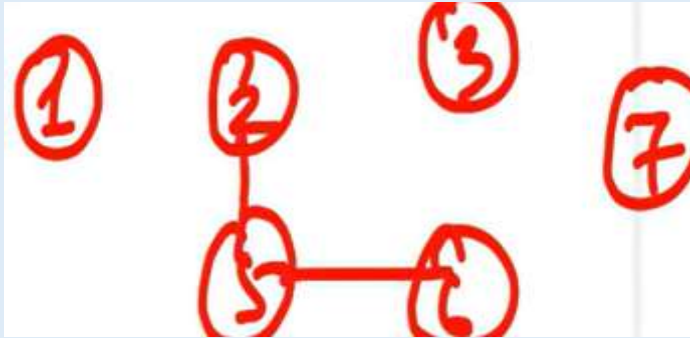
2 → 5, 1

5 → 6, 2

Min subset is:

$\{3, 1\}$

Among the vertices of 2 connections, we can choose any one. **Now we choose 1. Remove the edges connected to 1**



Vertex	Connected vertices	No. of Connections
1	[2,4]	2
4	[1]	1
2	[3,5,1]	3
5	[2,6,3]	3
3	[2,7,5,6]	4
6	[3,5]	2
7	[3]	1

Next Step: Now 5 is connected to 2

5 → 6, 2

Now we choose **5**. Remove the edges connected to **5**

Min subset is $\{3, 1, 5\}$

Vertex Cover : is the min subset of vertices which **covers all edges** in the given Graph G. In other words, Every edge's **any one end point** must be present as **one of the vertices** in **Min subset**.

Vertex Cover : Greedy Algorithm(1)

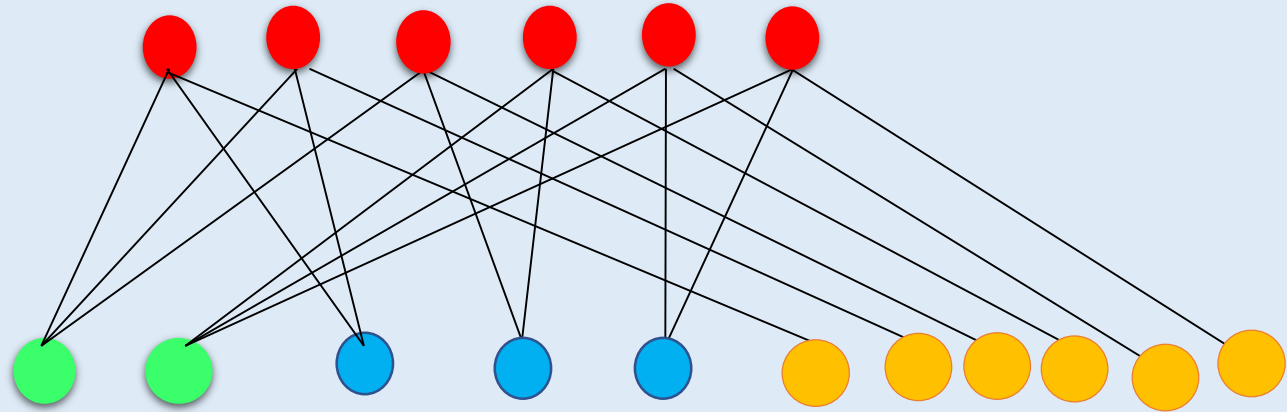
➤ **Idea:** Keep finding a vertex which covers the maximum number of edges.

Step 1: Find a vertex v with maximum degree.

Step 2: Add v to the solution and remove v and all its incident edges from the graph.

Step 3: Repeat until all the edges are covered.

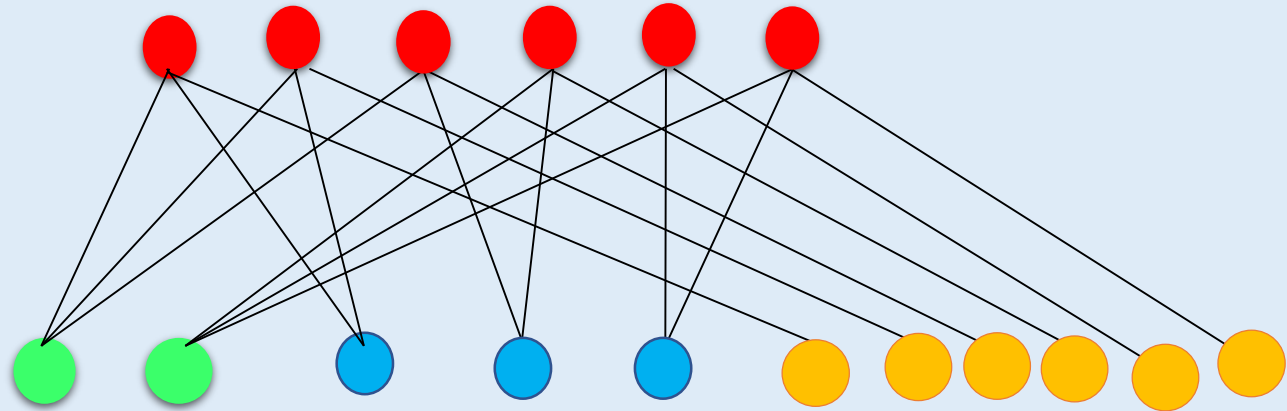
Greedy Algorithm(1): Analysis



Optimal Solution = 6, select all red vertices.

- Greedy approach does not always lead to the best approximation algorithm.

Greedy Algorithm(1): Analysis



Unfortunately if we select the vertices in following order, then we will get worst solution for this vertex cover problem-

- First we might choose all the green vertices.
- Then we might choose all the blue vertices.
- And then we might choose all the orange vertices. **Solution=11;**

Vertex Cover : Algorithm(2)

APPROX-VERTEX-COVER

1: $C \leftarrow \emptyset$;

2: $E' \leftarrow E$

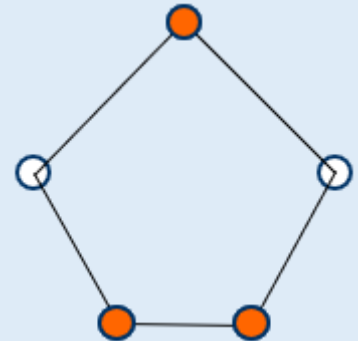
3: **while** $E' \neq \emptyset$; **do**

4: let (u, v) be an arbitrary edge of E'

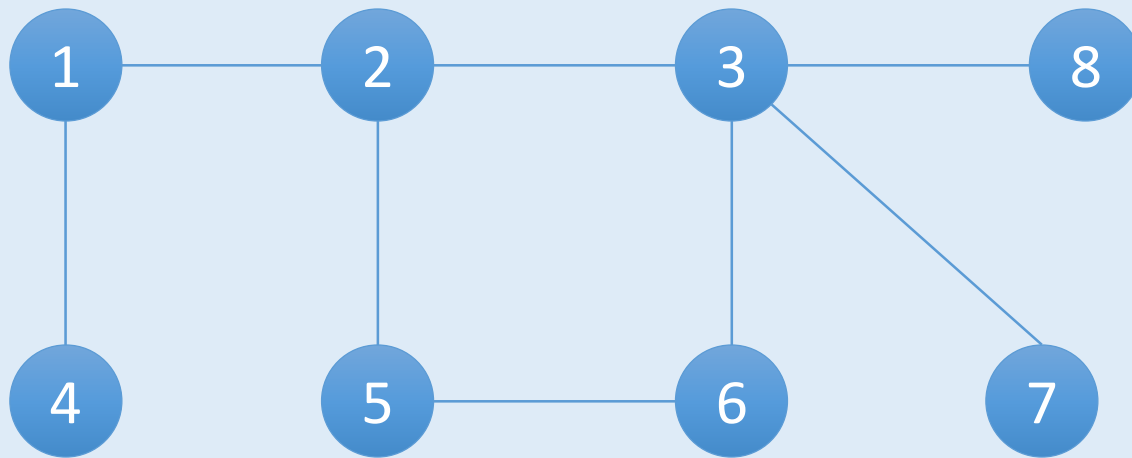
5: $C \leftarrow C \cup \{(u, v)\}$

6: remove from E' all edges incident on either u or v

7: **end while**



Algorithm(2): Example

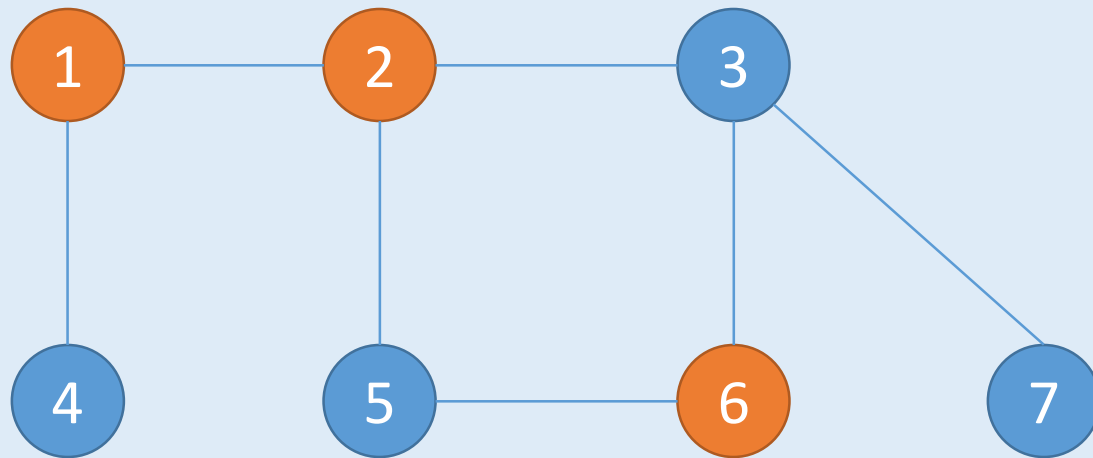


$C =$

1	2	3	6
---	---	---	---

$E' = \{(\underline{3,6}) (\underline{5,6}) (\underline{3,7}) (\underline{3,8})\}$

Algorithm(2): Example (Cont...)

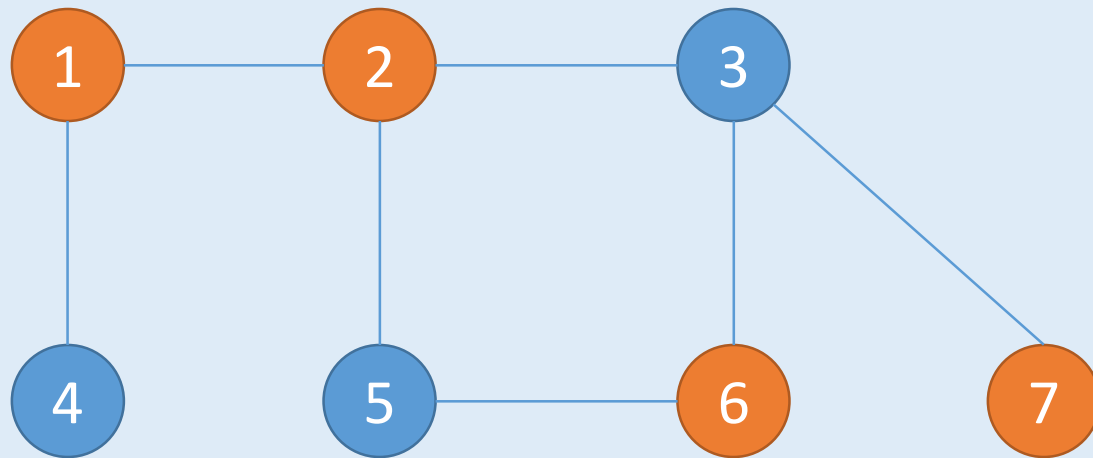


Are the red vertices a vertex-cover?

No..... why?

Edge (3, 7) is not covered by it.

Algorithm(2): Example (Cont...)



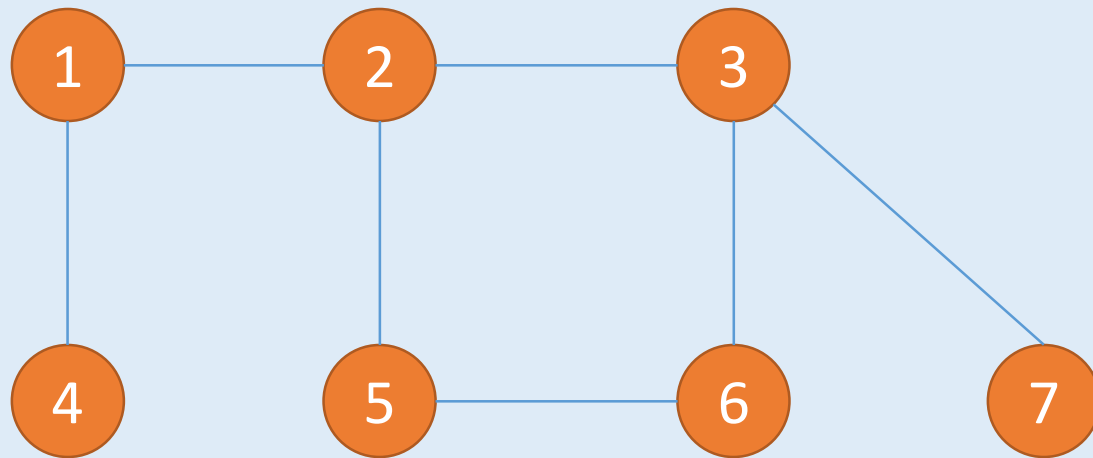
Are the red vertices a vertex-cover?

Yes

What is the size?

Size = 4

Algorithm(2): Example (Cont...)



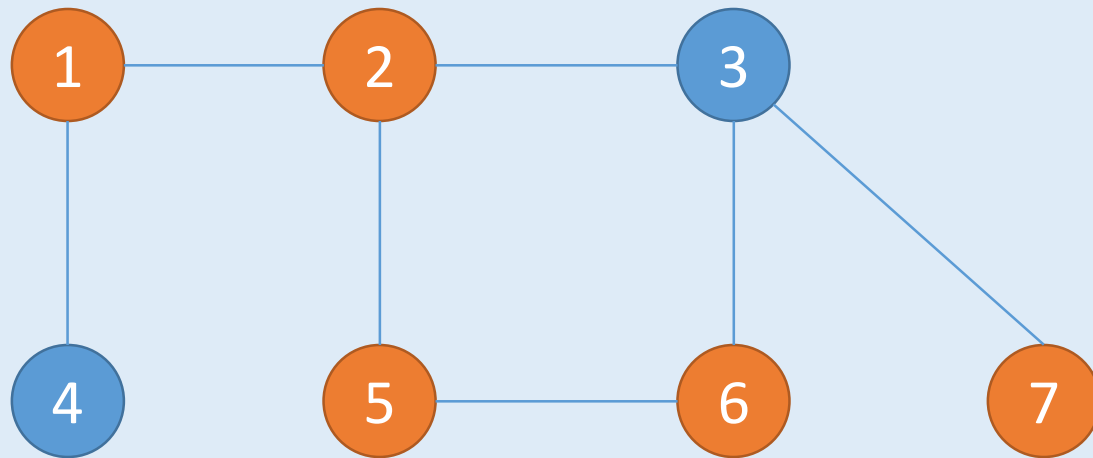
Are the red vertices a vertex-cover?

Of course.....

What is the size?

Size = 7

Algorithm(2): Example (Cont...)



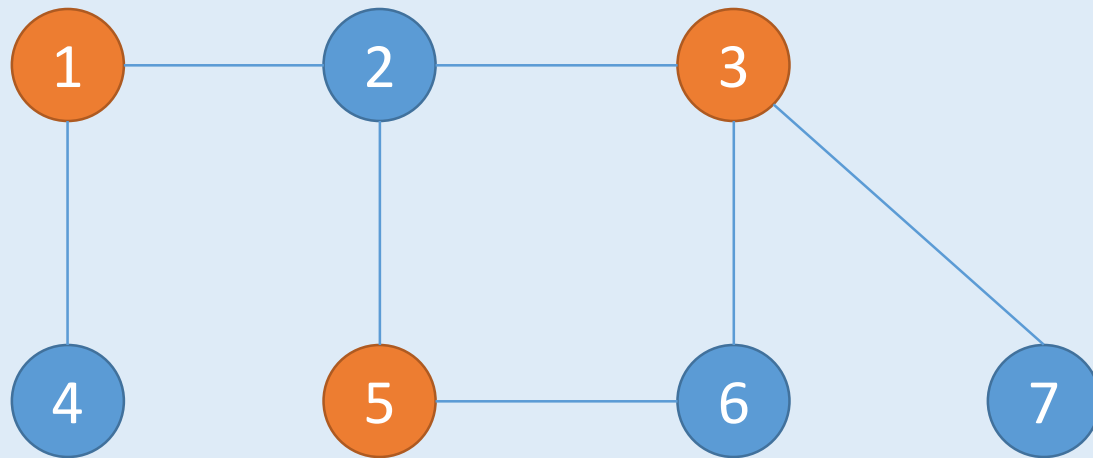
Are the red vertices a vertex-cover?

Yes

What is the size?

Size = 5

Algorithm(2): Example (Cont...)



Are the red vertices a vertex-cover?

Yes

What is the size?

Size = 3

Conclusion

- A set of **vertices** such that each edge of the graph is incident to at least one **vertex** of the set, is called the vertex cover.
- Greedy algorithm may or may not produce optimal solution.
- Approximation algorithm does not always guarantee optimal solution but its aim is to produce a solution which is as close as possible to the optimal solution.

Problem statement

➤ A network operator wants to place a minimum number of intrusion-detection sensors on hosts so every communication link is watched by at least one sensor. Each sensor watches all links incident on the host it sits on.

Question: can the operator place at most k sensors so that every network link is monitored?

Problem statement-Formal formulation

➤ Input: an undirected graph $G = (V, E)$ and an integer k .

Question: Does there exist a set $C \subseteq V$ with $|C| \leq k$ such that for every edge $\{u, v\} \in E$, $u \in C$ or $v \in C$?

Such a set C is called a **vertex cover** of size $\leq k$.

Problem statement-Formal formulation

➤ Input: an undirected graph $G = (V, E)$ and an integer k .

Question: Does there exist a set $C \subseteq V$ with $|C| \leq k$ such that for every edge $\{u, v\} \in E$, $u \in C$ or $v \in C$?

Such a set C is called a **vertex cover** of size $\leq k$.

Solution:

➤ Brute Force

➤ Greedy,

➤ Approximation algorithm

- **Complexity**
- **Membership in NP:** Yes — given a set C of vertices we can verify in polynomial time that every edge has an endpoint in C . So Vertex Cover $\in$ NP.
- **NP-Completeness:** Vertex Cover is a classical NP-Complete problem
- Try yourself:
- One standard way to show NP-hardness is via reductions from 3-SAT or by using the fact that Clique (another NP-Complete problem) reduces to Vertex Cover (complements of vertex sets in the graph complement correspond).
- Thus Vertex Cover is **NP-Complete**.

NP Hard or NP Complete?

- Try yourself:
- Subset sum Problem
- Graph coloring
- Relationship between P, NP, NP Hard and NP Complete

Thank You.....